Second Final Capstone Project

🎟 Marquee — A Live Events & Ticketing SaaS Backend

> **The second graduation project for the course.** Design and build the backend for **Marquee**, a multi-tenant events-and-ticketing platform in the spirit of Eventbrite / Ticketmaster / DICE — the kind of system where thousands of buyers hit a popular on-sale at the same second and the platform must sell every ticket **exactly once**.
>
> This is an **architecture, planning, and requirements** document — the brief a senior backend architect hands a developer. It contains **no implementation code**. Your job is to build it, applying concepts from **every phase of the course** where they naturally fit.
>
> **Companion to [Capstone 1 — Orbit].** Where Orbit stressed real-time *collaboration* and multi-tenant workflow, Marquee stresses **high-concurrency correctness under load** — inventory that must never oversell, a fair virtual queue, and a check-in path that validates a ticket once and only once. Same course, a different and equally demanding shape.

---

0. How to Use This Capstone

- **Scope:** A single motivated developer can build the **core** (Milestones 1–6) as a portfolio-grade backend; Milestone 7–8 and the Stretch Features are the "extra mile."
- **Standard:** Production-shaped. Layered architecture, real database with migrations, real auth, real-time, tested, containerized, deployable.
- **The promise:** Marquee is deliberately designed so that **every major concept in the syllabus appears where it genuinely belongs** — not bolted on. Section 3 (Syllabus Coverage Map) is the contract.
- **The signature challenge:** getting the **purchase path** right — no overselling under a flash-sale stampede, holds that expire, idempotent checkout, and a fair waiting room. Nail that and you've demonstrated senior-level backend judgement.
- **Grade yourself** against the Evaluation Criteria (Section 15).

---

1. Project Overview

1.1 The product

Marquee is a **B2B2C SaaS** platform. On one side, **event organizers** (the tenants) create venues and events, define ticket types and prices, schedule on-sales, and watch sales in real time. On the other side, the public **discovers** events, joins a **waiting room** for hot on-sales, **holds and buys** tickets without ever double-selling a seat, and receives a scannable **QR ticket**. At the door, **scanners** check attendees in at high throughput, each ticket admitted exactly once. Organizers get analytics, payouts, refunds, promo codes, integrations (API keys + webhooks), and an **AI attendee-support assistant**.

1.2 The problem it solves

Selling tickets is deceptively hard: demand is **spiky** (a popular show sells out in seconds), inventory is **finite and must never oversell**, money and fulfillment must be **consistent** (a paid order must yield exactly the right tickets, and a failed payment must release the held inventory), and the **door** must validate thousands of scans quickly without letting one ticket in twice. Marquee is the backend that does all of this correctly, for many organizers at once.

1.3 Who uses it

- **Attendees** (the public) — discover events, queue, hold, buy, receive tickets, get support.
- **Organizers** (tenant staff: owners/admins/event managers/finance/scanners) — run events, inventory, sales, refunds, check-in.
- **Partner systems** — marketing tools, CRMs, and resale partners that call Marquee via **API keys** and receive **webhooks**.
- **Platform operators** (you) — run, monitor, and scale it in production.

1.4 Why it's a strong capstone

Marquee is **inherently multi-concern**: multi-tenancy + RBAC (auth), rich relational data (database), a **correctness-critical purchase path** under concurrency (transactions, atomic inventory, idempotency), a **virtual waiting room** and live dashboards (SSE/WebSockets), notifications and ticket delivery (background tasks), discovery at scale (caching, search, pagination), abuse protection (rate limiting), an AI support assistant (streaming), and a full production/testing/deploy story. It cannot be built as "a pile of CRUD endpoints" — the on-sale forces real engineering.

---

2. Core Domain Model (Mental Model)

```
Organizer (tenant, has staff + payout settings)
   └── Venue  (place, capacity, optional seat map)
         └── Event  (a show/date, lifecycle: draft→published→on_sale→sold_out→completed/
               └── Ticket Type  (GA / VIP / Early-Bird: price, quantity, sales window)
                     └── Inventory  (finite; decremented atomically; never oversold)
                           ├── Hold      (a short-lived reservation with a TTL, during c
                           ├── Order      (a confirmed purchase by an Attendee)
                           │     └── Ticket  (issued, unique QR code, admitted exactly o
                           └── Waiting Room  (a fair queue that admits buyers in control
Attendee (global user)  buys across many organizers; owns Orders + Tickets
Scanner (organizer staff)  validates Tickets at the door (idempotent check-in)
```

Everything an organizer owns is **scoped to that organizer** — strict tenant isolation is the platform's #1 invariant. The platform's #2 invariant: **inventory never goes negative** and **each ticket is sold once and admitted once.**

---

3. Syllabus Coverage Map — the Contract

This capstone is designed so each concept has a **natural** home. Build these features and you've applied the whole course.

Phase / Lesson	Concept	Where it lives in Marquee
P1 · 1–8	REST design, path/query/body params, `Field` validation, status codes, `HTTPException`, `Response`	Every resource endpoint; event/ticket-type/order CRUD with correct verbs, IDs, codes
P2 · 9	Pydantic validators (`field_` / `model_validator`), `model_config`	Sales-window rules (start < end), price ≥ 0, quantity ≤ capacity, promo-code math, seat-selection validity
P2 · 10–11	`response_model`, Create/Read/Update schema separation	Hide internal fields (payment refs, raw QR secrets, cost basis); public vs organizer event views
P2 · 12–13	Status codes, custom exception handlers, global error handling	Domain errors → consistent JSON envelope; sold-out (409), hold-expired (410), queue-not-admitted (425/429)
P2 · 14	Dependency injection (sub-deps, class-based, `yield`)	DB session, current user, current organizer/tenant resolver, pagination params, permission + queue-token checks
P2 · 15	Middleware	Request-ID + timing + tenant-context middleware; CORS; GZip
P2 · 16	`APIRouter`, tags, prefixes, nested routers	One router per module (`auth`, `events`, `inventory`, `checkout`, `scan`…), versioned under `/api/v1`
P2 · 17	Form data & file uploads	Event cover images + venue seat-map assets; organizer logo
P2 · 18	Headers & cookies	`Idempotency-Key` on checkout, `X-Queue-Token`, `X-Request-ID`, refresh-token cookie option
P2 · 19	Static/templates	Rendered ticket/receipt email templates; the QR-bearing ticket page/PDF
P3 · 20–21	SQLAlchemy 2.0, models, relationships	The relational schema (Section 6): O2M, M2M, association objects, self-referential (categories)
P3 · 22–23	DB session dependency, CRUD, `from_attributes`, schema/model separation	Repository/service layers; ORM→schema serialization
P3 · 24	Alembic migrations	Every schema change ships as a reversible migration; `upgrade head` builds the DB
P3 · 25	Async SQLAlchemy	Async engine/session throughout; `selectinload` for relationships
P3 · 26	SQLModel *(alt)*	Optional: build one bounded context (e.g. discovery) with SQLModel to compare

P3 · 27	Redis (+ Mongo optional)	Redis for **inventory holds, queue, cache, rate-limit, pub-sub**; optional Mongo for the high-volume scan log
P4 · 28	Async vs sync, `run_in_threadpool`	Async endpoints; offload blocking work (QR/PDF generation, image processing)
P4 · 29	Auth: bcrypt, OAuth2, JWT (access + refresh), RBAC, API keys	Attendee + organizer-staff auth; organizer RBAC; partner API keys
P4 · 30	Background tasks	Issue tickets, send confirmation emails + QR, reminders, webhook delivery, release expired holds
P4 · 31	WebSockets	Live **organizer sales dashboard** (tickets sold, revenue, remaining) as sales happen
P4 · 32	SSE / streaming	The **virtual waiting-room** stream (your position/ admission) + the **AI support** token stream
P4 · 33	CORS	Configured for the (hypothetical) web/mobile buyer + organizer clients
P4 · 34	Rate limiting	On-sale abuse protection (per IP/user/API-key), scan-endpoint limits, AI token budget
P4 · 35	Caching	Hot event pages, discovery results, remaining-inventory counts (with careful invalidation)
P4 · 36	Pagination	Cursor pagination for discovery + order history; offset for organizer admin tables
P4 · 37	Filtering/sorting/searching	Discovery: filter by city/date/category/price, search by text, whitelisted sort (date/price/popularity)
P4 · 38	i18n *(optional)*	Localized attendee emails/notifications from `Accept-Language`
P4 · 39	GraphQL *(optional)*	A read-only GraphQL surface for organizer analytics/ reporting
P5 · 40–43	Testing + coverage + CI	pytest, `TestClient`/`httpx.AsyncClient`, isolated test DB, `dependency_overrides`, **concurrency tests for no-oversell**, coverage gate in CI
P6 · 44	Production project structure	Layered `app/` (routers/services/repositories/models/ schemas/core/db)
P6 · 45	Config management	`pydantic-settings`, `.env`, dev/staging/prod
P6 · 46	Logging	Structured JSON logs + request-ID tracing
P6 · 47	Security best practices	HTTPS/HSTS, secure headers, parameterized queries, signed QR tokens, secret management, generic auth errors

P6 · 48	Performance	Fix N+1 (eager loading), connection pooling, indexes on discovery + scan lookups, cache hot paths
P6 · 49	Docker	Multi-stage Dockerfile; `docker-compose` (app + Postgres + Redis)
P6 · 50	Production server	Gunicorn + Uvicorn workers behind Nginx
P6 · 51	Deployment	PaaS / Cloud Run + managed Postgres/Redis; migrations at release
P6 · 52	CI/CD	GitHub Actions: test → build → migrate → deploy, gated on `main`
P6 · 53	Monitoring	`/health/live` + `/health/ready`, `/metrics` (Prometheus), Sentry, request tracing
P6 · 54	API versioning	`/api/v1`; documented deprecation path for `/api/v2`
P6 · 55	OpenAPI customization	Rich metadata, examples, tags, hidden internal routes, `servers` list
Bonus · 59	LLM/AI patterns	The AI attendee-support assistant: token streaming, per-user token budgets, grounded on event info
Bonus · 56–58	Microservices, events, gRPC *(stretch)*	Split scanning/notifications into services; event bus for `order.paid`; gRPC for the scan-validation service

4. User Roles & RBAC

Marquee has **two role scopes**: platform-level and organizer-level. Attendees are their own global role. A user's effective permission is the combination of scope + role.

4.1 Attendee (public buyer)

Aspect	Detail
Permissions	Browse/search events; join a waiting room; hold + buy tickets; view own orders/tickets; request refunds per policy; chat with AI support
Responsibilities	Their own account and purchases
Accessible endpoints	Public discovery; `me` profile; own orders/tickets; checkout; support
Restrictions	No access to organizer data, other attendees' orders, or any admin surface

4.2 Organizer roles

Role	Responsibilities	Can	Cannot
Owner	Owns the organizer account	Everything below + manage billing/ payout + delete organizer	(nothing restricted)
Admin	Runs the organizer day-to-day	Manage staff/roles, venues, events, ticket types, promos, integrations	Delete the organizer, change payout owner
Event Manager	Runs specific events	Create/edit assigned events, ticket types, on-sales; view that event's sales	Manage staff, payouts, or other managers' events
Finance	Money	View sales/revenue, issue refunds, manage payouts	Edit events or inventory
Scanner	Door staff	Validate/scan tickets for assigned events; view check-in stats	Everything else (no catalog/ sales/refund access)

4.3 Platform role

- **Super-admin** (internal ops): cross-tenant support, abuse handling, feature flags — a small, audited surface, hidden from public docs.

4.4 Machine identity

- **API keys** (per organizer, scoped, hashed at rest) authenticate **partner integrations** — not humans. Scopes like `events:read`, `orders:read`, `scan:write`, `webhooks:manage`, each with its own rate limit.

4.5 RBAC enforcement rules

- Every data-access dependency resolves **(current identity, current organizer, current event)** and checks membership + role before the handler runs.
- **Tenant isolation is absolute:** cross-organizer access returns **404** (never 403 — don't leak existence).
- Insufficient role within a visible resource returns **403**.
- Scanners are constrained to their **assigned events** only.

---

5. Functional Requirements by Module

Each module: **what** it does, **why** it exists, **who** uses it, **business rules**, **validations**, **permissions**.

5.1 Identity & Auth

- **What:** Attendee + staff registration, login (OAuth2 password flow → JWT **access + refresh**), refresh, logout, password reset, `me`.
- **Why:** Prove identity; everything else depends on it.
- **Who:** Everyone.
- **Rules:** bcrypt hashing; short-lived access + revocable refresh tokens; email verification before purchase (optional); one account can be both an attendee and organizer staff.

- **Validations:** email format + uniqueness; password strength (respect bcrypt 72-byte limit); token type checks.
- **Permissions:** Public (register/login); authenticated (refresh/me).

5.2 Organizers & Staff

- **What:** Create organizer, org profile + payout settings, invite staff, assign organizer roles, remove staff.
- **Why:** The tenant boundary and the team that runs events.
- **Who:** Owners/admins manage; staff view as permitted.
- **Rules:** Creator becomes Owner; must keep ≥1 Owner; invites expire.
- **Validations:** unique organizer slug; role ∈ allowed set; invite email format.

5.3 Venues & Seat Maps

- **What:** CRUD venues (address, geo, capacity); optional **seat maps** (sections/rows/seats) for reserved-seating events; seat-map asset upload.
- **Why:** Physical capacity constrains inventory; seat maps enable reserved seating.
- **Rules:** A venue belongs to one organizer (or is a shared/global venue); total ticket-type quantity may not exceed venue capacity for a general-admission event.
- **Validations:** capacity > 0; seat identifiers unique within a section.

5.4 Events

- **What:** Create/edit events; manage lifecycle (`draft → published → on_sale → sold_out → completed / cancelled`); schedule the on-sale time; cover-image upload; categories/tags.
- **Why:** The thing people buy tickets to.
- **Who:** Event managers/admins create; the public views published events.
- **Rules:** Only `published` events appear in discovery; `on_sale` requires a start time and at least one ticket type; cancelling triggers refunds; controlled transitions only.
- **Validations:** `starts_at` in the future at publish; venue belongs to the organizer.

5.5 Ticket Types & Inventory

- **What:** Define ticket types per event (name, price, currency, **quantity**, per-order limit, sales window); track **remaining inventory**.
- **Why:** The finite, sellable, correctness-critical resource.
- **Who:** Event managers/admins.
- **Rules: Inventory is finite and may never go negative;** the sum of active holds + sold ≤ quantity; per-order limits enforced; sales only within the window and while the event is `on_sale`.
- **Validations:** price ≥ 0; quantity ≥ 0; per-order limit ≥ 1; window start < end.
- **Permissions:** organizer-scoped write; public read of remaining availability (possibly bucketed/cached).

5.6 Discovery (public)

- **What:** Browse/search/filter/sort a catalog of published events; event detail with live availability.
- **Why:** How attendees find and evaluate events.

- **Rules:** Only `published`/`on_sale` events; results are tenant-agnostic (across organizers); availability numbers are cached with short TTL.
- **Validations:** whitelisted sort fields (date/price/popularity); bounded `limit`.

5.7 Waiting Room (virtual queue)

- **What:** For high-demand on-sales, admit buyers in **controlled batches**; each waiting user gets a **live position** (SSE) and, when admitted, a **queue token** required to checkout.
- **Why:** Protect inventory and infrastructure from a stampede; make the sale **fair** (first-come) instead of a thundering herd.
- **Rules:** Admission rate is configurable per event; a queue token is single-use, time-boxed, and required by the hold/checkout endpoints during an active on-sale; leaving the page forfeits position (with a grace window).
- **Validations:** valid, unexpired queue token; one active queue entry per user per event.

5.8 Holds & Checkout (the signature path)

- **What:** Place a **hold** on N tickets of a type (or specific seats) with a **TTL**; complete **checkout** (payment) to convert the hold into an **order + issued tickets**; holds auto-expire and return inventory.
- **Why:** Lets a buyer complete payment without others grabbing the same tickets, while guaranteeing no oversell.
- **Who:** Attendees (with a valid queue token during on-sales).
- **Rules:**
- Creating a hold **atomically** decrements available inventory (reserved); expiry **atomically** returns it.
- **Checkout is idempotent** (`Idempotency-Key`): a retried checkout never charges twice or issues duplicate tickets.
- Checkout is a **Saga**: confirm hold → capture payment → issue tickets + mark order paid → publish `order.paid`; on payment failure, **compensate** (release hold, mark order failed).
- Per-order and per-user purchase limits enforced; promo codes validated and applied server-side.
- **Validations:** hold not expired; quantity within limits; totals server-computed (never client-trusted); payment token present.

5.9 Orders, Payments & Refunds

- **What:** Order history; payment via a provider (model it — provider integration is out of scope); refunds (full/partial) per policy and on event cancellation.
- **Why:** The money and fulfillment record.
- **Rules:** A paid order yields exactly the right tickets; refunds void the corresponding tickets and return/settle funds; refunds are transactional and audited.
- **Validations:** refund ≤ refundable amount; only `valid`/`unused` tickets refundable per policy.

5.10 Tickets & QR

- **What:** Issue tickets with a **unique, signed QR code**; deliver by email (background); attendee views/downloads tickets.
- **Why:** The admission credential.

- **Rules:** Each ticket has a unique code and a status (`valid / used / refunded / void`); the QR encodes a **signed** token (tamper-proof), not raw PII.
- **Validations:** signature/HMAC verified on scan; ticket belongs to the scanned event.

5.11 Check-in / Scanning (high-throughput)

- **What:** Scanners validate a ticket's QR at the door; the first valid scan admits and marks it `used`; subsequent scans are rejected as **already used**.
- **Why:** Fast, correct admission; prevents ticket reuse/fraud.
- **Who:** Scanners assigned to that event.
- **Rules:** Validation is **idempotent and atomic** — under concurrent scans of the same code, exactly one succeeds; scan results are logged; supports an **offline/eventual** reconciliation path (record scans, dedupe on sync).
- **Validations:** signed QR valid; ticket status `valid`; event matches; scanner authorized for the event.

5.12 Promotions & Discounts

- **What:** Promo codes (percentage/fixed, usage caps, validity window, per-event or per-ticket-type).
- **Rules:** Applied and validated server-side at checkout; usage counters decrement atomically; expired/exhausted codes rejected.

5.13 Real-Time Dashboards

- **What:** Live **organizer sales dashboard** over WebSockets: tickets sold, revenue, remaining inventory, check-in counts, updating as events happen.
- **Rules:** Authenticated + organizer/event-scoped before `accept()`; multi-worker fan-out via **Redis Pub/Sub**.

5.14 AI Attendee Support

- **What:** A streamed AI assistant that answers attendee questions grounded in the event's own info ("is it refundable?", "what time do doors open?", "where do I park?"), and helps find events.
- **Why:** Deflects support load; a modern SaaS expectation.
- **Rules:** Per-user/plan **token budget** (429 when exhausted); the model only sees **public/permitted** event data; usage metered; graceful degradation if the AI service is down.

5.15 Integrations: API Keys & Webhooks

- **What:** Organizers mint scoped **API keys**; register outbound **webhooks** (`order.paid`, `event.sold_out`, `ticket.checked_in`, …) with HMAC signing, retry/backoff, and dead-lettering.
- **Rules:** Keys hashed at rest, shown once, rotatable; webhook delivery via background worker.

5.16 Notifications

- **What:** Ticket delivery, order confirmations, on-sale reminders, event-change/cancellation notices — via background tasks (+ optional i18n).
- **Rules:** Respect user preferences; batched reminders scheduled.

5.17 Analytics & Reporting

- **What:** Sales over time, conversion, top events, check-in rates — aggregate, **cached** endpoints for organizers.
- **Rules:** Organizer-scoped; heavy queries cached with invalidation on new sales.

5.18 Audit & Admin

- **What:** Append-only audit of sensitive actions (refunds, role changes, key mints, event cancellation); internal health/metrics and (hidden) operational routes.

---

6. Database Design

PostgreSQL, async SQLAlchemy, Alembic migrations. Every organizer-owned table is **tenant-scoped**. Redis holds ephemeral **holds, queue state, and counters** (with the durable order/ticket truth in Postgres).

6.1 Entities & relationships

Table	Key columns	Relationships / notes
users	id, email (unique), hashed_password, full_name, is_active, created_at	Global accounts (attendee and/or staff)
organizers	id, name, slug (unique), owner_id → users, payout_details, created_at	The tenant root
organizer_members	id, organizer_id → organizers, user_id → users, role, **unique(organizer_id, user_id)**	**Association object** (staff role)
staff_event_assignments	organizer_member_id → organizer_members, event_id → events, **unique pair**	Scanners/managers scoped to events (M2M)
venues	id, organizer_id → organizers (nullable if shared), name, address, geo_lat, geo_lng, capacity	O2M from organizer
seat_sections	id, venue_id → venues, name, capacity	Optional reserved seating
seats	id, section_id → seat_sections, row, number, **unique(section_id,row,number)**	Reserved-seat inventory
categories	id, name, parent_id → categories (**self-referential**)	Event taxonomy
events	id, organizer_id → organizers, venue_id → venues, title, description, status, category_id → categories, cover_image_key, on_sale_at, starts_at, created_at	The core sellable event

ticket_types	id, event_id → events, name, price, currency, quantity_total, quantity_sold, per_order_limit, sales_start, sales_end	**Finite inventory** — `quantity_sold ≤ quantity_total` invariant
holds	id, ticket_type_id → ticket_types, user_id → users, quantity, seat_ids (nullable), status, expires_at, created_at	Short-lived reservation (mirrored in Redis with TTL)
orders	id, user_id → users, organizer_id → organizers, event_id → events, status, subtotal, discount, total, currency, idempotency_key, created_at	Purchase record; **unique(idempotency_key, user_id)**
order_items	id, order_id → orders, ticket_type_id → ticket_types, seat_id (nullable), quantity, unit_price	Lines
tickets	id, order_item_id → order_items, event_id → events, code (unique), qr_signature, status [valid/used/refunded/void], attendee_id → users, created_at	The issued credential
payments	id, order_id → orders, amount, provider_ref, status	Payment record (provider modeled)
refunds	id, order_id → orders, amount, reason, status, created_at	Refund record
checkins	id, ticket_id → tickets, event_id → events, scanned_by → users, gate, result, scanned_at, **unique(ticket_id) for a successful admit**	High-volume; dedupe/ idempotent
waiting_room_entries	id, event_id → events, user_id → users, position, token, admitted_at, expires_at, **unique(event_id,user_id) active**	Fair queue (backed by Redis)
promo_codes	id, organizer_id → organizers, event_id (nullable), code, type, value, max_uses, used_count, valid_from, valid_to, **unique(organizer_id, code)**	Discounts
payouts	id, organizer_id → organizers, amount, currency, status, period	Organizer settlement
api_keys	id, organizer_id → organizers, name, key_hash, prefix, scopes, last_used_at, revoked_at	Machine identity, hashed
webhooks	id, organizer_id → organizers, url, secret, events, is_active	Outbound integrations
webhook_deliveries	id, webhook_id → webhooks, event_type, payload, status, attempts, next_retry_at	Retry/dead-letter tracking

ai_support_sessions / ai_messages	ids, user_id → users, event_id (nullable), role, content, input_tokens, output_tokens	AI assistant + usage
notifications	id, user_id → users, type, payload, is_read, created_at	Per-user inbox
audit_log	id, organizer_id, actor_id, action, target, created_at	Immutable security log
outbox_events	id, aggregate_id, type, payload, published_at	Reliable event publishing

6.2 Relationship summary

- **One-to-many:** organizer→venues→events→ticket_types; event→tickets; order→order_items→tickets; venue→sections→seats.
- **Many-to-many (association objects):** user↔organizer (`organizer_members` with role); staff↔event (`staff_event_assignments`).
- **Self-referential:** `categories.parent_id`.
- **Ephemeral (Redis) + durable (Postgres):** holds and queue live in Redis for TTL/atomicity; orders/tickets are the durable source of truth.

6.3 Constraints & indexes

- **Uniques:** email, organizer slug, ticket `code`, promo `(organizer, code)`, membership pairs, seat `(section,row,number)`, order idempotency key.
- **The core invariant:** `quantity_sold ≤ quantity_total` (enforced by atomic updates + a check constraint); tickets issued per paid order match order items exactly.
- **Foreign keys** with sensible on-delete (cancel/cascade for owned children; restrict where it would orphan money/tickets).
- **Indexes:** `events(status, starts_at)` and `events(category_id, city)` for discovery; `ticket_types(event_id)`; `tickets(code)` **for fast scan lookup**; `orders(user_id, created_at)`; `checkins(event_id, scanned_at)`; `holds(expires_at)` for the sweeper.
- **Check constraints:** non-negative quantities/prices; status enums; `sales_start < sales_end`.

7. API Design

Everything under **`/api/v1`**. JSON. Auth via `Authorization: Bearer <jwt>` (users) or `X-API-Key` (partners). Consistent error envelope: `{ "error_code": "...", "detail": "...", "request_id": "..." }`. Standard codes: 200/201/204, 400, 401, 403, 404, 409, 410, 422, 425, 429, 5xx.

> Representative endpoints grouped by module (not exhaustive; list endpoints support pagination + filtering where noted).

Auth

Method	Path	Auth	Notes
POST	`/auth/register`	public	201; unique email (409)
POST	`/auth/login`	public (form)	200 → access + refresh; 401 generic
POST	`/auth/refresh`	refresh token	new access token
POST	`/auth/logout`	user	revoke refresh token
GET	`/auth/me`	user	current profile

Organizers & Staff

Method	Path	Auth	Notes
POST	`/organizers`	user	creator = Owner
GET/PATCH	`/organizers/{id}`	staff/admin	tenant-scoped (404 outside)
POST	`/organizers/{id}/staff`	admin	invite staff
PATCH	`/organizers/{id}/staff/{uid}`	admin	change role; keep ≥1 owner (409)
POST	`/organizers/{id}/staff/{uid}/events`	admin	assign scanner/manager to events

Venues & Events

Method	Path	Auth	Notes
POST/ GET	`/organizers/{id}/venues`	admin/ manager	venue + optional seat map (upload)
POST	`/events`	manager+	create (draft)
PATCH	`/events/{id}`	manager+	edit; controlled lifecycle
POST	`/events/{id}/publish` · `/on-sale` · `/cancel`	manager+	transitions (cancel → refunds)
POST/ GET	`/events/{id}/ticket-types`	manager+	define inventory

Discovery (public)

Method	Path	Auth	Notes
GET	`/discover/events`	public	**search/filter/sort + cursor pagination**, cached
GET	`/discover/events/{id}`	public	detail + live availability (cached, short TTL)

Waiting Room, Holds & Checkout

Method	Path	Auth	Notes
POST	`/events/{id}/queue`	user	join waiting room
GET	`/events/{id}/queue/stream`	user	**SSE** live position + admission (returns a queue token)
POST	`/events/{id}/holds`	user (+queue token)	**atomic** hold with TTL; 409 sold-out, 425 not-yet-admitted
DELETE	`/holds/{id}`	user	release early
POST	`/checkout`	user	**`Idempotency-Key`**; Saga: pay → issue tickets; 410 if hold expired

Orders, Tickets, Refunds

Method	Path	Auth	Notes
GET	`/orders` · `/orders/{id}`	user	own orders (cursor)
GET	`/tickets` · `/tickets/{id}`	user	own tickets + QR
POST	`/orders/{id}/refund`	user/finance	policy-checked; voids tickets

Check-in (scanners)

Method	Path	Auth	Notes
POST	`/scan`	scanner	validate QR; **idempotent**; 200 admit / 409 already-used / 422 invalid
GET	`/events/{id}/checkin-stats`	scanner/ manager	live counts

Real-time, AI, Integrations, Admin

Method	Path	Auth	Notes
WS	`/ws/organizers/{id}/events/{eid}/dashboard`	manager+	live sales/revenue/remaining
POST	`/support/sessions`	user	start AI support
GET	`/support/sessions/{id}/stream`	user	**SSE token streaming**; token budget (429)

POST/GET/DELETE	`/organizers/{id}/api-keys`	admin	key shown once; hashed
POST/GET	`/organizers/{id}/webhooks`	admin	signed deliveries
GET	`/organizers/{id}/analytics`	manager+/finance	**cached** aggregates
GET	`/organizers/{id}/audit-log`	admin	paginated
GET	`/health/live` · `/health/ready` · `/metrics`	public/internal	Ops

Selected request/response + error expectations

- **Hold:** request `{ ticket_type_id, quantity, seat_ids? }` + `X-Queue-Token`; response `{ hold_id, expires_at, subtotal }`; errors 409 (sold out), 425 (not admitted from queue), 422 (over per-order limit).
- **Checkout:** request `{ hold_id, payment_token, promo_code? }` + `Idempotency-Key`; response `{ order_id, status, tickets:[{id, code}] }`; errors 410 (hold expired), 409 (idempotency fingerprint mismatch), 402/409 (payment failed → compensation).
- **Scan:** request `{ qr_token, gate }`; response `{ result: "admitted", attendee, ticket_type }`; errors 409 (already used), 422 (invalid/tampered), 403 (scanner not assigned).

---

8. Real-Time & Asynchronous Architecture

- **WebSockets (Lesson 31):** the organizer **sales dashboard**; a Connection Manager per event; **Redis Pub/Sub** fans sale/check-in events across workers.
- **SSE (Lesson 32):** the **waiting-room** stream (position + admission token) and the **AI support** token stream — one-way, auto-reconnecting.
- **Background tasks (Lesson 30):** issue tickets + generate QR/PDF, send confirmation/reminder emails, deliver webhooks (retry/backoff), and **sweep expired holds** back into inventory.
- **Redis:** authoritative for **hold TTLs, queue position/admission, remaining-inventory counters, rate-limit counters, and pub/sub** — the pieces that must be fast and atomic under a flash sale.
- **Reliable events (Bonus 57, optional):** an **outbox** publishes `order.paid`, `event.sold_out`, `ticket.checked_in`; consumers (webhooks/notifications) are idempotent.

---

9. Production Architecture

Concern	Approach (lesson)
Config	`pydantic-settings` + `.env`; dev/staging/prod; secrets from env/secret manager (45, 47)
Logging	Structured JSON logs, request-ID tracing via middleware (46)

Monitoring	`/health/live` + `/health/ready`, `/metrics` (Prometheus), Sentry, optional OpenTelemetry (53)
Caching	Discovery results, hot event pages, availability counts; TTL + invalidation on sale/inventory change (35)
Rate limiting	Per IP/user/API-key; strict on queue/hold/scan; AI token budget; Redis-backed for multi-worker (34)
Security	HTTPS/HSTS, secure headers, parameterized queries, **HMAC-signed QR tokens**, hashed API keys, least-privilege DB user, generic auth errors (47)
Performance	Eager-load to kill N+1, connection pooling, indexes on discovery + `tickets.code` scan lookup, cache hot paths, cursor pagination (48)
Docker	Multi-stage image; `docker-compose` for app + Postgres + Redis (49)
Server	Gunicorn + Uvicorn workers behind Nginx (50)
Deployment	PaaS/Cloud Run + managed Postgres/Redis; migrations at release (51)
CI/CD	GitHub Actions: lint/test/coverage → build → `alembic upgrade head` → deploy, gated on `main` (52)
Versioning	`/api/v1`; documented deprecation strategy for `/api/v2` (54)
Docs	Customized OpenAPI: metadata, examples, tags, hidden internal routes, `servers` list (55)

10. Folder Structure (Production, Layered)

```
marquee/
├── app/
│   ├── main.py                    # assembly: app, middleware, routers, exception hand
│   ├── core/                      # config, security (JWT/bcrypt, QR signing), logging
│   │   ├── config.py
│   │   ├── security.py
│   │   ├── logging.py
│   │   └── events.py              # event bus / publisher (outbox)
│   ├── db/                        # engine, async session, Base, get_db
│   ├── redis/                     # client, holds, queue, counters, pub/sub
│   ├── models/                    # SQLAlchemy models (one file per aggregate)
│   ├── schemas/                   # Pydantic Create/Read/Update per resource
│   ├── repositories/              # data-access (queries; no HTTP, no business rules)
│   ├── services/                  # business logic: inventory, holds, checkout saga, q
│   │                              #   scanning, refunds, pricing/promos, RBAC, AI
│   ├── api/
│   │   ├── deps.py                # current_user, current_organizer, permissions, queu
│   │   └── v1/
│   │       ├── routes/            # auth, organizers, venues, events, ticket_types, di
│   │       │                      #   queue, checkout, orders, tickets, scan, support,
│   │       │                      #   integrations, analytics, admin
```

```
│   │       └── ws.py                # websocket dashboard + connection manager
│   ├── workers/                     # background tasks + consumers (tickets, email, webh
│   ├── templates/                   # ticket/receipt emails, QR ticket page
│   ├── exceptions.py                # domain errors + handlers
│   └── i18n/                        # (optional) translation catalogs
├── alembic/                         # migrations (env.py, versions/)
├── tests/
│   ├── conftest.py                  # fixtures: test DB (StaticPool), client, auth, over
│   ├── unit/                        # services/repositories in isolation
│   ├── integration/                 # DB-backed
│   ├── concurrency/                 # NO-OVERSELL + double-scan stress tests
│   └── api/                         # endpoint + auth + RBAC tests
├── docker/
│   ├── Dockerfile                   # multi-stage
│   ├── docker-compose.yml           # app + postgres + redis
│   └── nginx.conf
├── .github/workflows/ci-cd.yml
├── .env.example
├── pyproject.toml / requirements.txt
├── alembic.ini
└── README.md
```

> **By-layer** here for clarity; a larger team may migrate to **by-feature** packages (Lesson 44). Keep it consistent.

---

11. Testing Strategy

Test kind	What it covers	Tools (lessons 40–43)
Unit	Services in isolation (pricing/promo math, hold TTL logic, lifecycle transitions, RBAC)	pytest, no HTTP
Integration	Repositories against a real test DB; transactions; migrations apply cleanly	isolated test DB (`StaticPool`), per-test isolation
API	Endpoints end-to-end (queue → hold → checkout → ticket → scan)	`TestClient` / `httpx.AsyncClient`
Concurrency (signature)	**No overselling** — many parallel holds/checkouts on a scarce ticket type resolve to exactly the available count; **double-scan** — parallel scans of one code admit once	async clients + parallel tasks against Redis+DB
Auth & RBAC	401/403/404 matrix per role; refresh; API-key scopes; **tenant isolation** (Organizer A can't touch Organizer B); scanner scoped to assigned events	`app.dependency_overrides`
Real-time	WS dashboard broadcast; SSE queue/AI stream framing	`TestClient` websocket + streaming

External services	Payment provider + AI + email mocked via dependency overrides/fakes	mocking
Coverage	Gate in CI (e.g. `--cov-fail-under=85`), `term-missing` to find gaps	pytest-cov

Fixtures: `client`, isolated `db_session`, a fake/ephemeral Redis, factories for organizers/events/ticket-types, and role-scoped auth headers. Every test independent and order-independent. **The no-oversell and single-admit tests are non-negotiable** — they prove the platform's core invariant.

---

12. Development Roadmap (Milestones)

Build incrementally; each milestone yields a working, testable slice.

Milestone 1 — Foundation & Auth

- **Objective:** Runnable layered app; attendees + staff can register, log in, refresh.
- **Modules:** project structure (44), config (45), logging (46), DB + Alembic (21–24), auth (29), middleware (15), error handling (13).
- **Complexity:** Medium. **Estimated time:** 10–14 h. **Depends on:** —. **Outcome:** authenticated skeleton with migrations, `/health`, structured logs.

Milestone 2 — Tenancy, Venues & RBAC

- **Objective:** Organizers, staff, roles, venues (+ optional seat maps).
- **Modules:** organizers/staff/venues, RBAC dependencies, tenant isolation, file uploads (17).
- **Complexity:** Medium-High. **Estimated time:** 12–16 h. **Depends on:** M1. **Outcome:** multi-tenant, role-gated org management; 404-vs-403 discipline.

Milestone 3 — Events & Ticket Inventory

- **Objective:** Events with lifecycle + ticket types with finite inventory.
- **Modules:** events, ticket types, validators (9), response models (10–11), lifecycle rules.
- **Complexity:** Medium-High. **Estimated time:** 12–16 h. **Depends on:** M2. **Outcome:** organizers can build sellable events with correct inventory constraints.

Milestone 4 — Discovery

- **Objective:** Public search/filter/sort/paginate + event detail with availability.
- **Modules:** discovery, filtering/sorting/search (37), pagination (36), caching (35).
- **Complexity:** Medium. **Estimated time:** 8–12 h. **Depends on:** M3. **Outcome:** a fast, cached public catalog.

Milestone 5 — Purchase Core (the hard one)

- **Objective:** Holds with TTL + idempotent checkout Saga + ticket/QR issuance, with **guaranteed no overselling**.

- **Modules:** holds, checkout (30/18), payments (modeled), tickets + signed QR, Redis atomics (27), transactions, background ticket issuance + email.
- **Complexity: High. Estimated time:** 20–28 h. **Depends on:** M3 (+M4). **Outcome:** correct, concurrency-safe buying; the platform's core invariant proven by concurrency tests.

Milestone 6 — Waiting Room, Real-Time & Scanning

- **Objective:** Virtual queue (SSE), live organizer dashboard (WS), and high-throughput idempotent check-in.
- **Modules:** waiting room (32), WS dashboard (31), scanning, Redis pub/sub, rate limiting (34).
- **Complexity: High. Estimated time:** 18–24 h. **Depends on:** M5. **Outcome:** fair on-sales, live sales visibility, and a single-admit door.

Milestone 7 — Refunds, Promos, Integrations & AI

- **Objective:** Refunds/cancellations, promo codes, API keys + webhooks, and the streamed AI support assistant.
- **Modules:** refunds, promos, API keys (29), webhooks (background + retry), AI streaming + token budgets (59), analytics (cached).
- **Complexity:** High. **Estimated time:** 14–20 h. **Depends on:** M5–M6. **Outcome:** money completeness, partner integrations, and a modern AI feature.

Milestone 8 — Test, Harden, Ship

- **Objective:** Full test suite, monitoring, containerize, CI/CD, deploy.
- **Modules:** testing (40–43), security (47), monitoring (53), Docker (49), server (50), deployment (51), CI/CD (52), versioning (54), OpenAPI (55).
- **Complexity:** High. **Estimated time:** 16–22 h. **Depends on:** all. **Outcome:** a tested, monitored, containerized backend deployed via an automated pipeline.

Time Summary

Estimates assume a developer who has just completed this course (solid intermediate level), simulating the external payment/AI providers rather than fully integrating them.

Milestone	Focus	Hours
M1	Foundation & Auth	10–14
M2	Tenancy, Venues & RBAC	12–16
M3	Events & Ticket Inventory	12–16
M4	Discovery	8–12
M5	Purchase Core (the hard one)	20–28
M6	Waiting Room, Real-Time & Scanning	18–24
M7	Refunds, Promos, Integrations & AI	14–20

M8	Test, Harden, Ship	16–22
Core (M1–6)	portfolio-ready	**~80–110 h**
Full (M1–8)	complete	**~120–165 h**

On a calendar (full build): ~12–16 weeks part-time (~10 h/week) · ~6–8 weeks at ~20 h/week · ~3–4 weeks full-time.

Adjust for: skill level (junior fresh out of the course ≈ 1.5–2×; experienced backend dev ≈ 0.6–0.7×); wiring a *real* payment provider and/or LLM instead of a simulated one (+8–15 h each); and polish level (90%+ coverage, load tests, pixel-perfect OpenAPI can add ~15–25%).

> **Note:** Marquee runs a bit longer than Orbit overall — its M5/M6 concurrency correctness (no-oversell holds, idempotent checkout Saga, single-admit scanning, plus the concurrency test suite) is the hardest stretch in either capstone. **Recommended:** build **Core (M1–6)** first, writing Milestone 5's concurrency tests *first* to drive the design, then decide whether M7–M8 are worth the extra ~40–55 h.

13. Stretch Features (Beyond the Syllabus)

- **Reserved seating** — interactive seat maps with per-seat holds and best-available assignment.
- **Dynamic** / **tiered pricing** — price changes by demand or time (early-bird tiers auto-advancing).
- **Secondary market** / **resale** — safe ticket transfer and resale with re-issued QR and fraud checks.
- **Microservices split (Bonus 56–58)** — extract **Scanning** and **Notifications** into services; an event bus (RabbitMQ/Kafka) for `order.paid`/`ticket.checked_in`; **gRPC** for the low-latency scan-validation service.
- **i18n** — localized attendee emails/notifications via `Accept-Language` (Lesson 38).
- **GraphQL analytics** — a read-only GraphQL surface for organizer reporting (Lesson 39).
- **Fraud/abuse detection** — bot detection on on-sales, velocity checks, device fingerprinting.
- **Offline-first scanning** — a documented sync protocol so door scanning survives spotty venue Wi-Fi, reconciling on reconnect.
- **Multi-currency + payouts ledger**, **feature flags**, **data export (GDPR)**.

---

14. Advanced-Feature Placement (Quick Reference)

Feature	Natural home in Marquee
JWT + refresh tokens	Auth module
API keys	Partner integrations
RBAC	Organizer + event permission dependencies; scanner scoping

Background tasks	Ticket/QR issuance, emails, webhooks, expired-hold sweeper
WebSockets	Live organizer sales dashboard
Streaming (SSE)	Waiting-room position + AI support
Caching	Discovery, hot event pages, availability counts
Pagination / filter / sort / search	Discovery + order history
Rate limiting	On-sale/queue/hold/scan protection + AI token budget
File uploads	Event covers, venue seat maps, organizer logos
Async DB + transactions	Everywhere; atomic holds, checkout, and check-in
Redis	Holds/TTL, queue, counters, pub-sub, rate limits
Middleware / DI / exception handling	Request context, permissions, error envelope

15. Evaluation Criteria (Grade Yourself)

- **The core invariant:** under concurrent load, inventory **never oversells** and each ticket is **sold once and admitted once** — proven by concurrency tests, not just claimed.
- **Architecture:** clean layering (routers → services → repositories → models); no business logic in routes; thin `main.py`.
- **Correctness:** RBAC + **tenant isolation** airtight; the checkout **Saga** compensates on payment failure; **idempotent** checkout and scan.
- **Data:** correct relationships (O2M, M2M, association objects, self-ref), constraints, indexes; migrations build from empty and downgrade; Redis vs Postgres responsibilities clear.
- **Real-time & async:** WS/SSE authenticated and multi-worker-safe (Redis); background work reliable; expired holds always returned to inventory.
- **Performance:** no N+1 on hot paths; discovery + scan lookups indexed and cached; bounded, paginated lists.
- **Security:** hashed secrets, **signed QR tokens**, parameterized queries, secure headers, least privilege, generic auth errors, no secrets in logs/responses.
- **Testing:** meaningful unit/integration/API/RBAC **and concurrency** tests; isolated DB; coverage gate green in CI.
- **Production:** config by environment, structured logs, health + metrics, Dockerized, migrations-at-deploy, CI/CD, versioned + documented API.

🎓 Course Complete (Take Two)

If you build Marquee to this brief, you'll have shipped a **multi-tenant, high-concurrency, real-time, AI-assisted ticketing platform** — one that sells finite inventory correctly under a flash-sale stampede, runs a

fair queue, delivers signed tickets, admits each one exactly once at the door, and does it all authenticated, tested, observable, containerized, and deployed. That is the full arc of the course pointed at a different and unforgiving problem: **correctness under load.**

Between **Orbit** (real-time collaboration + workflow) and **Marquee** (high-concurrency commerce + inventory), you have two production-shaped graduation projects that together prove you can architect, secure, test, and operate real backend systems. Pick the one that excites you — or build both. **Now go build it.** 🚀

➡ Where to Go From Here

- Build Marquee milestone-by-milestone; treat each milestone as its own PR with tests and a green CI run.
- Write **Milestone 5's concurrency tests first** — let "no oversell" drive the design of holds and checkout.
- Revisit any lesson whose concept feels shaky when you reach the milestone that uses it (the Coverage Map in Section 3 is your index).
- Publish it: a deployed Marquee with a clear README and a load-test write-up ("sold 5,000 tickets with zero oversell under N concurrent buyers") is a standout backend-interview portfolio piece.